

Practical 7

Write a program that prints the addresses of code, global, static, heap, and stack variables. Analyze the Linux process address space layout and explain each memory segment.

Make use of the `/proc//maps` file and memory analysis tools to study the memory organization of a running process. Prepare a report explaining virtual memory mappings.

Program

```
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>

int global_var = 100;

void sampleFunction(){
    printf("Inside sampleFunction()\n");
}

int main(){
    int stack_var = 50;
    static int static_var = 200;

    int *heap_var = (int *)malloc(sizeof(int));

    if (heap_var == NULL){
        printf("Memory allocation failed!\n");
        return 1;
    }

    *heap_var = 300;

    printf("\n==== Linux Process Memory Layout =====\n");

    printf("Address of Code Segment (sampleFunction): %p\n",
        (void *)sampleFunction);

    printf("Address of Global Variable: %p\n",
        (void *)&global_var);

    printf("Address of Static Variable: %p\n",
        (void *)&static_var);

    printf("Address of Heap Variable: %p\n",
        (void *)heap_var);

    printf("Address of Stack Variable: %p\n",
        (void *)&stack_var);

    printf("\n==== /proc/self/maps =====\n");

    FILE *fp = fopen("/proc/self/maps", "r");
    if (fp == NULL){
        perror("fopen");
        free(heap_var);
        return 1;
    }
    char line[256];
    while (fgets(line, sizeof(line), fp)){
        printf("%s", line);
    }
    fclose(fp);
    free(heap_var);
    return 0;
}
```

Output

```
sai776421@Dra:~$ nano Practical7.c
sai776421@Dra:~$ gcc Practical7.c
sai776421@Dra:~$ ./a.out

===== Linux Process Memory Layout =====
Address of Code Segment (sampleFunction): 0x60518f3e3249
Address of Global Variable: 0x60518f3e6010
Address of Static Variable: 0x60518f3e6014
Address of Heap Variable: 0x60519d0a9010
Address of Stack Variable: 0x7ffce7821fdc

===== /proc/self/maps =====
60518f3e2000-60518f3e3000 r--p 00000000 08:30 1931 /home/sai776421/a.out
60518f3e3000-60518f3e4000 r-xp 00001000 08:30 1931 /home/sai776421/a.out
60518f3e4000-60518f3e5000 r--p 00002000 08:30 1931 /home/sai776421/a.out
60518f3e5000-60518f3e6000 r--p 00002000 08:30 1931 /home/sai776421/a.out
60518f3e6000-60518f3e7000 rw-p 00003000 08:30 1931 /home/sai776421/a.out
60519d0a9000-60519d0ca000 rw-p 00000000 00:00 0 [heap]
77674ba00000-77674ba28000 r--p 00000000 08:30 41990 /usr/lib/x86_64-linux-gnu/libc.so.6
77674ba28000-77674bbc0000 r-xp 00028000 08:30 41990 /usr/lib/x86_64-linux-gnu/libc.so.6
77674bbc0000-77674bc0e000 r--p 001c0000 08:30 41990 /usr/lib/x86_64-linux-gnu/libc.so.6
77674bc0e000-77674bc12000 r--p 0020e000 08:30 41990 /usr/lib/x86_64-linux-gnu/libc.so.6
77674bc12000-77674bc14000 rw-p 00212000 08:30 41990 /usr/lib/x86_64-linux-gnu/libc.so.6
77674bc14000-77674bc21000 rw-p 00000000 00:00 0
77674bc73000-77674bc76000 rw-p 00000000 00:00 0
77674bc7b000-77674bc7d000 rw-p 00000000 00:00 0
77674bc7d000-77674bc81000 r--p 00000000 00:00 0 [vvar]
77674bc81000-77674bc83000 r--p 00000000 00:00 0 [vvar_vclock]
77674bc83000-77674bc85000 r--p 00000000 00:00 0 [vdso]
77674bc85000-77674bc86000 r--p 00000000 08:30 41987 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
77674bc86000-77674bcb5000 r-xp 00001000 08:30 41987 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
77674bcb5000-77674bcc0000 r--p 00030000 08:30 41987 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
77674bcc0000-77674bcc2000 r--p 0003b000 08:30 41987 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
77674bcc2000-77674bcc3000 rw-p 0003d000 08:30 41987 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
77674bcc3000-77674bcc4000 rw-p 00000000 00:00 0
7ffce7803000-7ffce7824000 rw-p 00000000 00:00 0 [stack]
```

Practical 8

Develop a program using malloc(), calloc(), realloc(), and free(). Monitor memory allocation behavior and identify memory leaks using Valgrind.

Develop a program that demonstrates Copy-on-Write behavior after a fork(). Analyze memory usage before and after modifying data in the child process.

Program 8.1

```

GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>

int main(){
    int i, n = 5;

    int *arr1 = (int *)malloc(n * sizeof(int));

    if (arr1 == NULL){
        printf("malloc failed\n");
        return 1;
    }

    printf("Memory allocated using malloc:\n");
    for (i = 0; i < n; i++){
        arr1[i] = i + 1;
        printf("%d ", arr1[i]);
    }
    printf("\n");

    int *arr2 = (int *)calloc(n, sizeof(int));
    if (arr2 == NULL){
        printf("calloc failed\n");
        free(arr1);
        return 1;
    }

    printf("\nMemory allocated using calloc:\n");
    for (i = 0; i < n; i++){
        printf("%d ", arr2[i]);
    }
    printf("\n");

    n = 10;
    arr1 = (int *)realloc(arr1, n * sizeof(int));

    if (arr1 == NULL){
        printf("realloc failed\n");
        free(arr2);
        return 1;
    }

    printf("\nAfter realloc (size increased to 10):\n");
    for (i = 5; i < n; i++){
        arr1[i] = i + 1;
    }

    for (i = 0; i < n; i++){
        printf("%d ", arr1[i]);
    }
    printf("\n");
    free(arr1);
    free(arr2);
    printf("\nMemory successfully freed.\n");
    return 0;
}

```

OUTPUT

```

sai776421@Dra:~$ nano Practical8.1.c
sai776421@Dra:~$ gcc Practical8.1.c
sai776421@Dra:~$ ./a.out
Memory allocated using malloc:
1 2 3 4 5

Memory allocated using calloc:
0 0 0 0 0

After realloc (size increased to 10):
1 2 3 4 5 6 7 8 9 10

Memory successfully freed.

```

Program 8.2

```
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

#define SIZE 1000000

int main(){
    int *data;
    int i;

    data = (int *)malloc(SIZE * sizeof(int));

    if (data == NULL){
        perror("malloc");
        return 1;
    }

    for (i = 0; i < SIZE; i++){
        data[i] = i;
    }

    printf("Parent Process PID = %d\n", getpid());
    printf("Address of data = %p\n", (void *)data);
    pid_t pid = fork();

    if (pid < 0){
        perror("fork");
        free(data);
        return 1;
    }

    if (pid == 0){
        printf("\nChild Process PID = %d\n", getpid());
        printf("Before modification: data[0] = %d\n", data[0]);

        data[0] = 9999;

        printf("After modification: data[0] = %d\n", data[0]);

        sleep(10);
        free(data);
        exit(0);
    }else{
        wait(NULL);

        printf("\nParent Process:\n");
        printf("data[0] = %d\n", data[0]);

        free(data);
    }

    return 0;
}
```

OUTPUT

```
sai776421@Dra:~$ nano Practical8.2.c
sai776421@Dra:~$ gcc Practical8.2.c
sai776421@Dra:~$ ./a.out
Parent Process PID = 781
Address of data = 0x76ef6722f010

Child Process PID = 782
Before modification: data[0] = 0
After modification: data[0] = 9999

Parent Process:
data[0] = 0
```

Practical 9

Implement a file copy utility using low-level Linux file I/O system calls (open, read, write, lseek, close). Compare its performance with standard library functions (fopen, fread, fwrite).

Develop a program that redirects standard input and output using file descriptors and dup2(). Demonstrate how shells implement I/O redirection.

Program 9

```
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>

#define BUFFER_SIZE 4096

int main(int argc, char *argv[]){
    int src, dest;
    ssize_t bytesRead, bytesWritten;
    char buffer[BUFFER_SIZE];

    if (argc != 3){
        printf("Usage: %s <source_file> <destination_file>\n", argv[0]);
        return 1;
    }

    src = open(argv[1], O_RDONLY);
    if (src < 0){
        perror("Error opening source file");
        return 1;
    }

    dest = open(argv[2], O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (dest < 0){
        perror("Error creating destination file");
        close(src);
        return 1;
    }

    while ((bytesRead = read(src, buffer, BUFFER_SIZE)) > 0){
        bytesWritten = write(dest, buffer, bytesRead);

        if (bytesWritten != bytesRead){
            perror("Write error");
            close(src);
            close(dest);
            return 1;
        }
    }

    if (bytesRead < 0){
        perror("Read error");
    }

    printf("File copied successfully.\n");
    close(src);
    close(dest);

    return 0;
}
```

Output

```
sai776421@Dra:~$ nano Practical9.c
sai776421@Dra:~$ gcc Practical9.c
sai776421@Dra:~$ ./a.out
Usage: ./a.out <source_file> <destination_file>
sai776421@Dra:~$ nano cat.txt
sai776421@Dra:~$ nano cat.txt
sai776421@Dra:~$ gcc cat.txt
/usr/bin/x86_64-linux-gnu-ld.bfd:cat.txt: file format not recognized; treating as linker script
/usr/bin/x86_64-linux-gnu-ld.bfd:cat.txt:1: syntax error
collect2: error: ld returned 1 exit status
sai776421@Dra:~$ gcc Practical9.c
sai776421@Dra:~$ ./a.out
Usage: ./a.out <source_file> <destination_file>
sai776421@Dra:~$ cat Poutput.txt
cat: Poutput.txt: No such file or directory
sai776421@Dra:~$ nano cat.txt
sai776421@Dra:~$ cat cat.txt
Hello Operating Systems
Linux File Descriptors
dup2 Demonstration
sai776421@Dra:~$ nano poutput.txt
sai776421@Dra:~$ cat poutput.txt
hello llllllllllllllllllllllllllllllllllllllllllllllll
sai776421@Dra:~$ ls -l
Practical2.c
Practical7.c
Practical8.1
Practical8.1.c
Practical8.2.c
Practical9.c
Robin.c
Week1.c
Week2.c
Week3.c
Week4.c
Week5.c
Week6
Week6.1.c
Week6.2.c
Week6.c
Wprogram5
Wprogram5.c
Zombie
Zombie.c
a.out
cat.txt
copy.txt
poutput.txt
sample.txt
week2.c
sai776421@Dra:~$ ./Practical9 cat.txt Poutput.txt\
>
-bash: ./Practical9: No such file or directory
sai776421@Dra:~$ ./Practical9 cat.txt Poutput.txt
-bash: ./Practical9: No such file or directory
sai776421@Dra:~$ ./a.out cat.txt Poutput.txt
File copied successfully.
sai776421@Dra:~$ gcc Practical9.c -o Practical9
sai776421@Dra:~$ ./Practical9 cat.txt Poutput.txt
File copied successfully.
```

Program 9.1

```
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>

#define BUFFER_SIZE 4096

int main(int argc, char *argv[]){
    FILE *src, *dest;
    size_t bytesRead;
    char buffer[BUFFER_SIZE];

    if (argc != 3){
        printf("Usage: %s <source_file> <destination_file>\n", argv[0]);
        return 1;
    }

    src = fopen(argv[1], "rb");
    if (src == NULL){
        perror("Source file");
        return 1;
    }

    dest = fopen(argv[2], "wb");
    if (dest == NULL){
        perror("Destination file");
        fclose(src);
        return 1;
    }

    while ((bytesRead = fread(buffer, 1, BUFFER_SIZE, src)) > 0){
        fwrite(buffer, 1, bytesRead, dest);
    }

    printf("File copied successfully.\n");
    fclose(src);
    fclose(dest);

    return 0;
}
```

Output

```
sai776421@Dra:~$ nano Practical9.1.c
sai776421@Dra:~$ gcc Practical9.1.c
sai776421@Dra:~$ ./a.out
Usage: ./a.out <source_file> <destination_file>
sai776421@Dra:~$ ./ Practical9.1 cat.txt poutput.txt
-bash: ./: Is a directory
sai776421@Dra:~$ ./a.out cat.txt Poutput.txt
File copied successfully.
sai776421@Dra:~$ ./Practical9.1 cat.txt poutput.txt
-bash: ./Practical9.1: No such file or directory
sai776421@Dra:~$ nano Practical9.1.c
sai776421@Dra:~$ gcc Practical9.1.c -o Practical9.1
sai776421@Dra:~$ ./Practical9.1 cat.txt Poutput.txt
File copied successfully.
```

Program 9.2

```
GNU nano 8.7.1
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>

int main(){
    int fd_in, fd_out;
    char buffer[256];
    fd_in = open("input.txt", O_RDONLY);

    if (fd_in < 0){
        perror("Input file");
        return 1;
    }

    fd_out = open("output.txt",
                  O_WRONLY | O_CREAT | O_TRUNC,
                  0644);

    if (fd_out < 0){
        perror("Output file");
        return 1;
    }

    dup2(fd_in, STDIN_FILENO);
    dup2(fd_out, STDOUT_FILENO);

    while (fgets(buffer, sizeof(buffer), stdin) != NULL){
        printf("%s", buffer);
    }

    close(fd_in);
    close(fd_out);

    return 0;
}
```

Output

```

sai776421@Dra:~$ ./Practical9.2.c
-bash: ./Practical9.2.c: Permission denied
sai776421@Dra:~$ gcc Practical9.2.c -o Practical9.2
sai776421@Dra:~$ ls -l Practical9.2
-rwxr-xr-x 1 sai776421 sai776421 16264 Sep 24 06:49 Practical9.2
sai776421@Dra:~$ ./Practical9.2
sai776421@Dra:~$ ./Practical9.2.c
-bash: ./Practical9.2.c: Permission denied
sai776421@Dra:~$ gcc Practical9.2.c -o Practical9.2
sai776421@Dra:~$ ./Practical9.2
sai776421@Dra:~$ ls -l output.txt
-rw-r--r-- 1 sai776421 sai776421 66 Sep 24 06:50 output.txt
sai776421@Dra:~$ cat output.txt
Hello Operating Systems
Linux File Descriptors
dup2 Demonstration
sai776421@Dra:~$ ls -l input.txt
ls -l output.txt
cat output.txt
-rw-r--r-- 1 sai776421 sai776421 66 Sep 24 06:38 input.txt
-rw-r--r-- 1 sai776421 sai776421 66 Sep 24 06:50 output.txt
Hello Operating Systems
Linux File Descriptors
dup2 Demonstration
sai776421@Dra:~$ ./Practical9.2
sai776421@Dra:~$ cat output.txt
Hello Operating Systems
Linux File Descriptors
dup2 Demonstration
sai776421@Dra:~$ ./Practical9.2 input.txt output.txt

```

Practical 10

Investigate inode structures using commands such as `ls -li`, `stat`, and `find`. Create hard links and symbolic links and explain their impact on inode allocation.

Design a program using `mmap()` to perform file reading and writing. Compare memory-mapped I/O with traditional read/write operations in terms of efficiency and implementation complexity.

Program 10

```
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <sys/mman.h>
#include <unistd.h>
#include <string.h>

int main(){
    int fd;
    char *map;

    fd = open("mmapfile.txt", O_RDWR | O_CREAT, 0666);

    if(fd < 0){
        perror("open");
        return 1;
    }

    ftruncate(fd, 100);
    map = mmap(NULL, 100,
               PROT_READ | PROT_WRITE,
               MAP_SHARED,
               fd, 0);

    if(map == MAP_FAILED){
        perror("mmap");
        close(fd);
        return 1;
    }

    strcpy(map, "Hello from Memory Mapped File!");
    printf("Data Written:\n%s\n", map);
    munmap(map, 100);
    close(fd);

    return 0;
}
```

Output

```

sai776421@Dra:~$ nano Practical10.c
sai776421@Dra:~$ gcc Practical10.c -o Practical10
sai776421@Dra:~$ ./Practical10
Data Written:
Hello from Memory Mapped File!
sai776421@Dra:~$ cat mmapfile.txt
Hello from Memory Mapped File!sai776421@Dra:~$ |

```

Program 10.1

```

GNU nano 8.7.1
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <string.h>

int main(){
    int fd;
    char buffer[100];

    fd = open("rwfile.txt",
              O_RDWR | O_CREAT | O_TRUNC,
              0666);

    write(fd, "Hello using read/write!", 23);
    lseek(fd, 0, SEEK_SET);
    read(fd, buffer, 23);
    buffer[23] = '\0';
    printf("Data Read:\n%s\n", buffer);
    close(fd);

    return 0;
}

```

Output

```

Hello from Memory Mapped File!sai776421@Dra:~$ nano Practical10.1.c
sai776421@Dra:~$ gcc Practical10.1.c -o Practical10.1
sai776421@Dra:~$ ./Practical10.1
-bash: ./: Is a directory
sai776421@Dra:~$ ./Practical10.1.c
-bash: ./Practical10.1.c: Permission denied
sai776421@Dra:~$ gcc output.txt
/usr/bin/x86_64-linux-gnu-ld.bfd:output.txt: file format not recognized; treating as linker script
/usr/bin/x86_64-linux-gnu-ld.bfd:output.txt:1: syntax error
collect2: error: ld returned 1 exit status
sai776421@Dra:~$ cat input.txt
Hello Operating Systems
Linux File Descriptors
dup2 Demonstration
sai776421@Dra:~$ ./Practical10.1.c input.txt
-bash: ./Practical10.1.c: Permission denied
sai776421@Dra:~$ gcc Practical10.1.c -o Practical10.1
sai776421@Dra:~$ ls -l Practical10.1
-rwxr-xr-x 1 sai776421 sai776421 16224 Sep 24 08:24 Practical10.1
sai776421@Dra:~$ ./Practical10.1 input.txt
Data Read:
Hello using read/write!
sai776421@Dra:~$ gcc output.txt
/usr/bin/x86_64-linux-gnu-ld.bfd:output.txt: file format not recognized; treating as linker script
/usr/bin/x86_64-linux-gnu-ld.bfd:output.txt:1: syntax error
collect2: error: ld returned 1 exit status
sai776421@Dra:~$ |

```

Practical 11

Develop a multithreaded counter application using POSIX threads (pthread_create, pthread_join). Demonstrate race conditions by allowing multiple threads to update a shared variable concurrently.

Modify the previous program using mutex locks (pthread_mutex_lock, pthread_mutex_unlock) and compare the outputs with the race-condition version. Analyze the impact of synchronization.

Program 11

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

#define NUM_THREADS 5
#define INCREMENTS 100000

long long counter = 0;
void *increment_counter(void *arg){
    for (int i = 0; i < INCREMENTS; i++){
        counter++;
    }
    pthread_exit(NULL);
}

int main(){
    pthread_t threads[NUM_THREADS];
    for (int i = 0; i < NUM_THREADS; i++){
        pthread_create(&threads[i], NULL, increment_counter, NULL);
    }

    for (int i = 0; i < NUM_THREADS; i++){
        pthread_join(threads[i], NULL);
    }

    printf("Expected Counter Value = %d\n",
          NUM_THREADS * INCREMENTS);

    printf("Actual Counter Value    = %lld\n", counter);
    return 0;
}
```

Output

```

sai776421@Dra:~$ nano Practical11.c
sai776421@Dra:~$ gcc Practical11.c -o Practical11
sai776421@Dra:~$ ./Practical11
Expected Counter Value = 500000
Actual Counter Value   = 442568
sai776421@Dra:~$ |

```

Program 11.1

```

GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

#define NUM_THREADS 5
#define INCREMENTS 100000

long long counter = 0;
pthread_mutex_t lock;
void *increment_counter(void *arg){
    for (int i = 0; i < INCREMENTS; i++){
        pthread_mutex_lock(&lock);
        counter++;
        pthread_mutex_unlock(&lock);
    }
    pthread_exit(NULL);
}

int main(){
    pthread_t threads[NUM_THREADS];
    pthread_mutex_init(&lock, NULL);

    for (int i = 0; i < NUM_THREADS; i++){
        pthread_create(&threads[i], NULL, increment_counter, NULL);
    }

    for (int i = 0; i < NUM_THREADS; i++){
        pthread_join(threads[i], NULL);
    }

    pthread_mutex_destroy(&lock);
    printf("Expected Counter Value = %d\n",
        NUM_THREADS * INCREMENTS);

    printf("Actual Counter Value   = %lld\n", counter);
    return 0;
}

```

Output

```
sai776421@Dra:~$ nano Practical11.1.c
sai776421@Dra:~$ gcc Practical11.1.c -o Practical11.1
sai776421@Dra:~$ ./Practical11.1
Expected Counter Value = 500000
Actual Counter Value   = 500000
sai776421@Dra:~$ |
```

Practical 12

Implement the Producer-Consumer problem using counting semaphores and POSIX threads. Evaluate synchronization correctness and system throughput under different buffer sizes.

Design a deadlock scenario involving multiple threads competing for shared resources. Demonstrate deadlock occurrence, identify the four necessary conditions for deadlock, and implement a deadlock prevention strategy using resource ordering.

Program 12

```
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define BUFFER_SIZE 5
#define ITEMS 20

int buffer[BUFFER_SIZE];
int in = 0, out = 0;
sem_t empty, full;
pthread_mutex_t mutex;
void *producer(void *arg){
    int item;
    for (item = 1; item <= ITEMS; item++){
        sem_wait(&empty);
        pthread_mutex_lock(&mutex);
        buffer[in] = item;
        printf("Producer produced: %d at %d\n", item, in);
        in = (in + 1) % BUFFER_SIZE;
        pthread_mutex_unlock(&mutex);
        sem_post(&full);
        sleep(1);
    }
    pthread_exit(NULL);
}

void *consumer(void *arg){
    int item;
    for (int i = 1; i <= ITEMS; i++){
        sem_wait(&full);
        pthread_mutex_lock(&mutex);
        item = buffer[out];
        printf("Consumer consumed: %d from %d\n", item, out);
        out = (out + 1) % BUFFER_SIZE;
        pthread_mutex_unlock(&mutex);
        sem_post(&empty);
        sleep(2);
    }
    pthread_exit(NULL);
}

int main(){
    pthread_t prod, cons;

    sem_init(&empty, 0, BUFFER_SIZE);
    sem_init(&full, 0, 0);
    pthread_mutex_init(&mutex, NULL);

    pthread_create(&prod, NULL, producer, NULL);
    pthread_create(&cons, NULL, consumer, NULL);

    pthread_join(prod, NULL);
    pthread_join(cons, NULL);

    sem_destroy(&empty);
    sem_destroy(&full);
    pthread_mutex_destroy(&mutex);

    return 0;
}
```

Output

```
sai776421@Dra:~$ nano Practical12.c
sai776421@Dra:~$ gcc Practical12.c -o Practical12
sai776421@Dra:~$ ./Practical12
Producer produced: 1 at 0
Consumer consumed: 1 from 0
Producer produced: 2 at 1
Consumer consumed: 2 from 1
Producer produced: 3 at 2
Producer produced: 4 at 3
Consumer consumed: 3 from 2
Producer produced: 5 at 4
Producer produced: 6 at 0
Consumer consumed: 4 from 3
Producer produced: 7 at 1
Producer produced: 8 at 2
Consumer consumed: 5 from 4
Producer produced: 9 at 3
Producer produced: 10 at 4
Consumer consumed: 6 from 0
Producer produced: 11 at 0
Consumer consumed: 7 from 1
Producer produced: 12 at 1
Consumer consumed: 8 from 2
Producer produced: 13 at 2
Consumer consumed: 9 from 3
Producer produced: 14 at 3
Consumer consumed: 10 from 4
Producer produced: 15 at 4
Consumer consumed: 11 from 0
Producer produced: 16 at 0
Consumer consumed: 12 from 1
Producer produced: 17 at 1
Consumer consumed: 13 from 2
Producer produced: 18 at 2
Consumer consumed: 14 from 3
Producer produced: 19 at 3
Consumer consumed: 15 from 4
Producer produced: 20 at 4
Consumer consumed: 16 from 0
Practical12.cConsumer consumed: 17 from 1
^C
```

Program 12.1

```
GNU nano 8.7.1
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t resource1;
pthread_mutex_t resource2;

void *thread1(void *arg){
    pthread_mutex_lock(&resource1);
    printf("Thread 1 locked Resource 1\n");
    sleep(1);
    printf("Thread 1 waiting for Resource 2\n");
    pthread_mutex_lock(&resource2);
    printf("Thread 1 acquired Resource 2\n");
    pthread_mutex_unlock(&resource2);
    pthread_mutex_unlock(&resource1);
    return NULL;
}

void *thread2(void *arg){
    pthread_mutex_lock(&resource2);
    printf("Thread 2 locked Resource 2\n");
    sleep(1);
    printf("Thread 2 waiting for Resource 1\n");
    pthread_mutex_lock(&resource1);
    printf("Thread 2 acquired Resource 1\n");
    pthread_mutex_unlock(&resource1);
    pthread_mutex_unlock(&resource2);
    return NULL;
}

int main(){
    pthread_t t1, t2;

    pthread_mutex_init(&resource1, NULL);
    pthread_mutex_init(&resource2, NULL);

    pthread_create(&t1, NULL, thread1, NULL);
    pthread_create(&t2, NULL, thread2, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    pthread_mutex_destroy(&resource1);
    pthread_mutex_destroy(&resource2);

    return 0;
}
```

Output

```
sai776421@Dra:~$ gcc Practical12.1.c -o Practical12.1
sai776421@Dra:~$ ./Practical12.1
Thread 2 locked Resource 2
Thread 1 locked Resource 1
Thread 2 waiting for Resource 1
Thread 1 waiting for Resource 2
|
```

Program 12.2

```
GNU nano 8.7.1
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t resource1;
pthread_mutex_t resource2;

void *thread1(void *arg){
    pthread_mutex_lock(&resource1);
    printf("Thread 1 locked Resource 1\n");
    sleep(1);
    pthread_mutex_lock(&resource2);
    printf("Thread 1 locked Resource 2\n");
    printf("Thread 1 working...\n");
    pthread_mutex_unlock(&resource2);
    pthread_mutex_unlock(&resource1);

    return NULL;
}

void *thread2(void *arg){
    pthread_mutex_lock(&resource1);
    printf("Thread 2 locked Resource 1\n");
    sleep(1);
    pthread_mutex_lock(&resource2);
    printf("Thread 2 locked Resource 2\n");
    printf("Thread 2 working...\n");
    pthread_mutex_unlock(&resource2);
    pthread_mutex_unlock(&resource1);

    return NULL;
}

int main(){
    pthread_t t1, t2;

    pthread_mutex_init(&resource1, NULL);
    pthread_mutex_init(&resource2, NULL);

    pthread_create(&t1, NULL, thread1, NULL);
    pthread_create(&t2, NULL, thread2, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    pthread_mutex_destroy(&resource1);
    pthread_mutex_destroy(&resource2);

    return 0;
}
```

Output

```
sai776421@Dra:~$ nano Practical12.2.c
sai776421@Dra:~$ gcc ^C
sai776421@Dra:~$ gcc Practical12.2.c -o ^C
sai776421@Dra:~$ gcc Practical12.2.c -o Practical12.2
sai776421@Dra:~$ ./Practical12.2
Thread 1 locked Resource 1
Thread 1 locked Resource 2
Thread 1 working...
Thread 2 locked Resource 1
Thread 2 locked Resource 2
Thread 2 working...
sai776421@Dra:~$ |
```